Lecture 5: MapReduce Design Patterns I (Basic)

DSL351 / DSP352

Amit Kumar Dhar

ED1, 406A
amitkdhar@iitbhilai

January 14, 2026

Introduction to Design Patterns

Why Design Patterns?

- MapReduce is a low-level paradigm (Map $\rightarrow$ Shuffle $\rightarrow$ Reduce).
- Common data processing problems recur frequently:
  - "Find the average rating."
  - "Filter out bad records."
  - "Sort the data globally."
- **Design Patterns** are standardized, reusable templates for solving these common problems.
- They help in writing robust, scalable, and readable code.

Pattern Categories

We will cover the following categories of basic patterns today:

1. **Summarization Patterns:** Calculating aggregate statistics (min, max, count, avg) or indices.
2. **Filtering Patterns:** Selecting a subset of records based on conditions.
3. **Organization Patterns:** Reorganizing data layout (binning, partitioning).

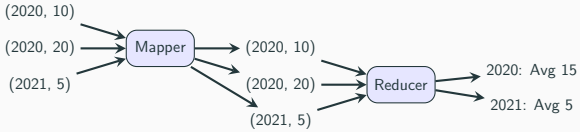
Summarization Patterns

1. Numerical Summarization

Goal: Calculate aggregate statistics over your data.

- **Examples:** Word count, Average salary by department, Max temperature by year.
- **Structure:**
  - **Mapper:** Emits keys (groups) and numerical values.
  - **Reducer:** Aggregates the values for each key.
  - **Combiner:** Highly recommended to reduce network traffic.

Concept: Numerical Summarization



Numerical Summarization: Mapper

```
public class AverageMapper extends
    Mapper<Object, Text, IntWritable, CountAverageTuple> {

    private IntWritable outYear = new IntWritable();
    private CountAverageTuple outCountAverage = new CountAverageTuple();

    public void map(Object key, Text value, Context context) ... {
        Map<String, String> parsed = transformXmlToMap(value.toString());

        String year = parsed.get("CreationDate").substring(0, 4);
        float score = Float.parseFloat(parsed.get("Score"));

        outYear.set(Integer.parseInt(year));
        outCountAverage.setCount(1);
        outCountAverage.setSum(score);

        context.write(outYear, outCountAverage);
    }
}
```

Custom Writable for Averages

To calculate averages, we need both *sum* and *count*. We create a custom Writable.

```
public class CountAverageTuple implements Writable {
    private float count = 0;
    private float sum = 0;

    public void readFields(DataInput in) throws IOException {
        count = in.readFloat();
        sum = in.readFloat();
    }
    public void write(DataOutput out) throws IOException {
        out.writeFloat(count);
        out.writeFloat(sum);
    }
    // Getters and Setters...
}
```

Numerical Summarization: Reducer

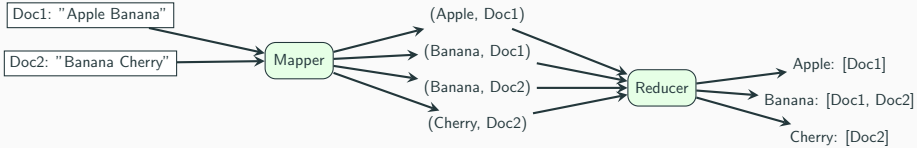
```
1  public class AverageReducer extends
2      Reducer<IntWritable, CountAverageTuple, IntWritable, CountAverageTuple> {
3
4      private CountAverageTuple result = new CountAverageTuple();
5
6      public void reduce(IntWritable key, Iterable<CountAverageTuple> values,
7          Context context) ... {
8
9          float sum = 0;
10         float count = 0;
11
12         for (CountAverageTuple val : values) {
13             sum += val.getSum();
14             count += val.getCount();
15         }
16         result.setCount(count);
17         result.setSum(sum); // Or calculate average here
18         context.write(key, result);
19     }
20 }
```

2. Inverted Index Pattern

Goal: Generate an index from dataset content to its location.

- **Analogy:** The index at the back of a textbook (Term $\rightarrow$ Page Numbers).
- **Big Data Use Case:** Search Engines. Mapping terms to URL/Document IDs.
- **Structure:**
  - **Mapper:** Parses document. Emits ‘(Word, DocID)‘.
  - **Reducer:** Collects all ‘DocIDs‘ for a ‘Word‘. Emits ‘(Word, List<DocID>)‘.

Concept: Inverted Index



Inverted Index: Mapper

```java
public class IndexMapper extends Mapper<Object, Text, Text, Text> {
    private Text word = new Text();
    private Text docId = new Text();

    public void map(Object key, Text value, Context context) ... {
        // Assuming input is "DocID\tContent"
        String[] parts = value.toString().split("\t");
        String id = parts[0];
        String content = parts[1];

        docId.set(id);

        for (String token : content.split(" ")) {
            word.set(token);
            context.write(word, docId);
        }
    }
}
```

Inverted Index: Reducer

```
1  public class IndexReducer extends Reducer<Text, Text, Text, Text> {
2      private Text result = new Text();
3
4      public void reduce(Text key, Iterable<Text> values, Context context) ... {
5          StringBuilder sb = new StringBuilder();
6          boolean first = true;
7
8          for (Text id : values) {
9              if (!first) sb.append(",");
10             sb.append(id.toString());
11             first = false;
12         }
13
14         result.set(sb.toString());
15         context.write(key, result);
16     }
17 }
```

Filtering Patterns

3. Filtering Patterns Overview

Goal: Filter out records that don't meet a criterion.

- Doesn't change the data, just selects a subset.
- **Map-Only Job:** No Reducer is needed.
- **Set Reducer to 0:** `job.setNumReduceTasks(0)`
- Map output is directly written to HDFS.

Simple Filter: Code

```
public class GrepMapper extends Mapper<Object, Text, NullWritable, Text> {

    public void map(Object key, Text value, Context context) ... {
        if (value.toString().contains("ERROR")) {
            // Emitting NullWritable as key effectively ignores it
            context.write(NullWritable.get(), value);
        }
    }
}

// In Driver:
job.setNumReduceTasks(0);
```

4. Bloom Filter Pattern

Problem: Checking membership in a massive set (e.g., "Is this URL in our blacklist?").

- Storing the whole blacklist in memory is impossible.
- Disk lookups are too slow.

Solution: Bloom Filter.

- A probabilistic data structure.
- **False Positives:** Possible (says "Yes" but actually "No").
- **False Negatives:** Impossible (if it says "No", it's definitely "No").
- Extremely space-efficient.

Bloom Filter Logic

1. **Training Phase:** Create the Bloom Filter from the "hot list" (e.g., blacklist) and serialize it to a file.
2. **Distribution:** Use Hadoop's **DistributedCache** to send this file to all nodes.
3. **Filtering Phase (Mapper):**
   - Load Bloom Filter into memory ‘setup()‘.
   - In ‘map()‘, check ‘filter.test(record)‘.
   - If ‘true‘ (maybe match) or ‘false‘ (definite mismatch), handle accordingly.

Bloom Filter: Mapper

```java
public class BloomFilterMapper extends Mapper<Object, Text, Text, NullWritable> {
    private BloomFilter<String> filter;

    protected void setup(Context context) {
        URI[] files = context.getCacheFiles();
        // Load filter from file...
        filter = BloomFilter.read(new FileInputStream(files[0]));
    }

    public void map(Object key, Text value, Context context) {
        String comment = value.toString();
        if (filter.mightContain(comment)) {
            context.write(value, NullWritable.get());
        }
    }
}
```

5. Distinct Pattern

Goal: Remove duplicate records.

- **Approach 1 (Exact):** Use standard MapReduce.
  - Map: ‘write(record, NullWritable)‘
  - Reduce: ‘write(key, NullWritable)‘ (Groups identical keys, outputs once).
- **Approach 2 (Approximate):** Bloom Filters (for massive datasets where exact precision isn’t critical but speed is).

Distinct: Exact Reducer

```
1  public class DistinctReducer extends
2      Reducer<Text, NullWritable, Text, NullWritable> {
3
4      public void reduce(Text key, Iterable<NullWritable> values, Context context)
5          throws IOException, InterruptedException {
6
7          // The framework groups identical keys.
8          // We just write the key once.
9          context.write(key, NullWritable.get());
10     }
11 }
```

Organization Patterns

6. Binning Pattern

Goal: Organize records into different output files/folders based on a value.

- **Example:** Separate log data into ‘ERROR‘, ‘WARN‘, ‘INFO‘ folders.
- **Standard MR:** Only one output directory.
- **Solution:** `MultipleOutputs` class.
- Can be done in Map-only (no shuffling) or Reducer.

The `MultipleOutputs` Class

- Standard MapReduce produces files like `part-m-00000` in a single directory.
- `MultipleOutputs` allows a task to write to multiple files or even different directories.
- **Key Capabilities:**
  - **Named Outputs:** Pre-define output labels in the Job configuration.
  - **Dynamic Paths:** Determine the target file path at runtime based on record content.
  - **Flexibility:** Supports different key/value types for different outputs.
  - `mos.write(namedOutput, key, value, baseOutputPath)`
- **Arguments Explained:**
  - `namedOutput`: Logical name for the output (defined in Driver).
  - `key value`: The record to be written to the output.
  - `baseOutputPath`: The specific file or folder name (relative to the job's main output directory).
- **Usage:** Initialize in `setup()`, write using `mos.write()`, and close in `cleanup()`.

Binning: Mapper with MultipleOutputs

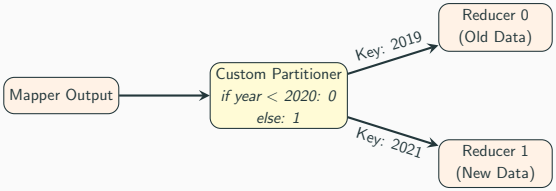
```java
public class BinningMapper extends Mapper<Object, Text, Text, NullWritable> {
    private MultipleOutputs<Text, NullWritable> mos;

    protected void setup(Context context) {
        mos = new MultipleOutputs<>(context);
    }

    public void map(Object key, Text value, Context context) ... {
        String log = value.toString();
        if (log.contains("ERROR")) {
            mos.write("bins", value, NullWritable.get(), "error-logs");
        } else if (log.contains("WARN")) {
            mos.write("bins", value, NullWritable.get(), "warn-logs");
        }
    }

    protected void cleanup(Context context) {
        mos.close();
    }
}
```

7. Partitioning Pattern

Goal: Control which Reducer processes which record.

- **Default:** ‘HashPartitioner‘ $\rightarrow$ $hash(key)\%numReducers$.
- **Problem:** What if we want all data for "Region A" in one file (Reducer 1) and "Region B" in another (Reducer 2)?
- **Solution:** Custom ‘Partitioner‘.

Concept: Partitioning



Partitioning: Custom Code

```java
public class YearPartitioner extends Partitioner<IntWritable, Text> {

    @Override
    public int getPartition(IntWritable key, Text value, int numPartitions) {
        // key is Year
        int year = key.get();

        if (numPartitions < 2) return 0; // Safety check

        if (year < 2020) {
            return 0; // Goes to Reducer 0
        } else {
            return 1; // Goes to Reducer 1
        }
    }
}

// In Driver:
job.setPartitionerClass(YearPartitioner.class);
job.setNumReduceTasks(2);
```

8. Total Order Sorting

Goal: Produce a globally sorted output file (or set of files).

- **Problem:** Hadoop sorts keys *within* a reducer, but not *across* reducers.
- **Solution:** Use a Partitioner that respects order.
- **Example:**
  - Reducer 0: Keys A-G
  - Reducer 1: Keys H-M
  - Reducer 2: Keys N-Z
- Requires understanding the distribution of keys (sampling) to create balanced partitions.

Summary

Recap

Pattern Type	Key Techniques
Summarization	Combiners, Custom Writables for aggregations
Inverted Index	Emit (Value $\rightarrow$ Key), Gather IDs in Reducer
Filtering	Map-only jobs, Bloom Filters (DistributedCache)
Binning	`MultipleOutputs` in Mapper
Partitioning	Custom `Partitioner` logic

Questions?